

Manual OpenFoam

October 5, 2026

1 Mallado (Mesh)

En esta sección veremos cómo se generan las mallas en OpenFOAM y mencionaremos algunas herramientas externas, como la importación de geometrías desde FreeCAD.

1.1 Generación

En OpenFOAM la estructura de malla se conoce como `polyMesh`. Para empezar a construir cualquier geometría necesitamos entender de qué está compuesta.

- **Puntos:** están colocados en el espacio 3D, definidos por un vector (x, y, z) , y se enumeran en el orden en que aparecen comenzando desde cero. En la lista no puede haber puntos con las mismas coordenadas ni algún punto que no esté contenido en alguna cara. Los denotaremos por p_n , donde n es un número natural incluyendo el cero.
- **Caras:** es una lista ordenada de puntos, donde los puntos se referencian mediante su posición en el archivo. Para ir colocando los puntos de una cara tenemos que usar la regla de la mano derecha y, en ese orden, asignarlos. Existen dos tipos de caras:
 - **Caras internas:** están conectadas a dos celdas.
 - **Caras de frontera:** pertenecen a una sola celda y su vector normal debe apuntar hacia fuera del dominio computacional.

- **Celdas:** es una lista de caras en orden arbitrario. Deben ser continuas (no deben encimarse), convexas (su centro debe estar dentro de la celda), cerradas y ortogonales.
- **Frontera:** es el conjunto de caras que delimitan el dominio y que se agrupan en parches (*patches*) con un nombre y un tipo determinados.

Ahora que sabemos de qué se compone el mallado, podemos empezar a crear formas. Para esto tenemos que saber que las formas están definidas por el orden en que colocamos los puntos; este orden debe hacer que las figuras no sean degeneradas o estén retorcidas. Para esto primero ocupamos una lista de puntos de esta manera:

```
vertices
(
  (-20.6 0 -0.5)
  (-20.6 25.4 -0.5)
  (0 -25.4 -0.5)
  (0 0 -0.5)
  (0 25.4 -0.5)
  (206 -25.4 -0.5)
  (206 0 -0.5)
);
```

Figure 1: Vértices del mallado

Una vez dada la lista de puntos para cualquier figura, tenemos que dar una lista de los puntos de esa figura. Debemos colocar estos puntos de manera que, si existe una arista entre dos puntos, estos aparezcan consecutivos en la lista.

Después tenemos que poner entre paréntesis cómo queremos que estas figuras sean divididas: a , b y c indican la división del bloque en cada dirección.

(a b c)

Veamos las formas que existen y cómo usarlas:

- **Hexaedros:** para definir un hexaedro lo hacemos con:

hex (p_1 p_2 p_3 ... p_8) (a b c)

- **Wedge:** `wedge (p1 p2 p3 ... p7) (a b c)`
- **Prisma:** `prism (p1 p2 p3 ... p6) (a b c)`
- **Pirámide:** `pyr (p1 p2 p3 ... p5) (a b c)`
- **Tetraedro:** `tet (p1 p2 p3 ... p4) (a b c)`
- **Arista de tetraedro:** `tetWedge (p1 p2 p3 ... p5) (a b c)`

Para colocar cualquiera de estos debemos ponerlos dentro de algo llamado `blocks` de esta manera:

```
blocks
(
  hex (0 1 2 3 4 5 6 7) (1 1 1)
);
```

En este bloque van todas las figuras.

1.2 Tipos de caras

Esto nos define el comportamiento que tendrán las caras de las formas. Estas están divididas en los siguientes tipos:

- **patch:** no contiene ninguna propiedad geométrica o topológica del mal-lado, pero sirve para *inlet* u *outlet*; no aporta información sobre la pared.
- **wall:** pared sólida dentro del dominio.
- **symmetryPlane:** replica lo que ocurre en el plano de simetría.
- **empty:** sirve para resolver mejor los casos en 2 dimensiones agregando la condición de **empty**.

Para colocar las caras debemos hacer lo siguiente:

```
boundary
(
  caras
{
```

```

        type wall;
        faces
        (
            (0 1 2 3)
            (1 3 4 5)
        );
    }
};

```

donde **caras** es cualquier nombre y en **type** puede ir cualquiera de los tipos de la lista anterior.

1.3 Compilar

Para poder ver lo que hemos hecho tenemos que tener lo siguiente: la escala, que usualmente va al inicio y se ve así: **convertToMeters x;** o **scale x;**, donde x es un número y esta es la escala de distancias. Después colocamos **vertices**, **blocks** y, por último, **boundary**.

Una vez que tenemos esto, podemos correr lo siguiente para compilar y visualizar:

```

blockMesh
paraFoam &

```

Aquí **blockMesh** compila y **paraFoam** abre el visualizador para ver la malla.

1.4 cfMesh

cfMesh es una librería de código abierto para generación automática de mallas, desarrollada por Creative Fields. A diferencia de **blockMesh**, que requiere que el usuario defina manualmente los bloques, **cfMesh** puede generar mallas a partir de una geometría de superficie (STL) de forma casi automática.

Sus utilidades principales son:

- **cartesianMesh**: genera una malla predominantemente hexaédrica a partir de una superficie triangulada. Es la opción más robusta y rápida para geometrías complejas.
- **tetMesh**: genera una malla tetraédrica, útil cuando la geometría tiene detalles muy pequeños que una malla hexaédrica no podría capturar bien.

- **pMesh**: genera una malla poliedral, que combina celdas de distintos tipos y suele reducir el número total de celdas.

El flujo de trabajo típico con **cfMesh** es:

1. Preparar un archivo STL con la superficie del dominio.
2. Crear un archivo **system/meshDict** con los parámetros de refinamiento y capas límite.
3. Ejecutar la utilidad correspondiente, por ejemplo: **cartesianMesh**.
4. Verificar la malla con **checkMesh**.

cfMesh no viene incluido en la distribución estándar de OpenFOAM; debe compilarse e instalarse por separado.

1.5 snappyHexMesh

snappyHexMesh es la herramienta nativa de OpenFOAM para generar mallas 3D complejas a partir de una malla de fondo hexaédrica creada con **blockMesh**. Su funcionamiento se basa en tres etapas consecutivas:

1. **Castellated mesh**: se parte de la malla de fondo y se eliminan las celdas que quedan fuera de la geometría (definida por un archivo STL). Las celdas cercanas a la superficie se refinan según los niveles especificados en **snappyHexMeshDict**.
2. **Snap**: las celdas que quedaron cortadas por la superficie se ajustan (*snap*) para que sus caras coincidan con la geometría. Aquí es donde se preservan las aristas vivas extraídas previamente con **surfaceFeatureExtract**.
3. **Add layers**: se añaden capas límite (*boundary layers*) cerca de la superficie para capturar correctamente los gradientes de velocidad en la pared, algo esencial para modelos de turbulencia de bajo número de Reynolds.

El flujo de trabajo típico es:

```
blockMesh
surfaceFeatureExtract
snappyHexMesh -overwrite
checkMesh
```

El archivo `system/snappyHexMeshDict` controla todos los parámetros: la geometría de entrada (`geometry`), los refinamientos locales (`refinementSurfaces`, `refinementRegions`), los parámetros de *snapping* y la configuración de capas. Es un archivo extenso y su correcta configuración es uno de los aspectos más delicados del preprocesamiento en OpenFOAM.

2 Solucionadores(solvers)

En esta sección veremos 4 solvers para flujo laminar, turbulento, los que están definidos para un número grande de eddy y otro de dos estados

2.1 icoFoam

Este solver es para fluidos incompresible y laminar, las ecuaciones son

$$\nabla \cdot u = 0$$

$$\frac{\partial}{\partial t}(u) + \nabla \cdot (u \otimes u) - \nabla \cdot (\nu \nabla u) = -\nabla p$$

Ahora para poderlo usar vamos a copiar el un tutorial por lo que en la terminal escribiremos lo siguiente

```
cd $FOAM_RUN
cp -r $FOAM_TUTORIALS/incompressible/icoFoam/cavity/cavity .
cd cavity
```

Ahora veamos que tenemos dentro de la carpeta. Primero podemos abrir la carpeta `system` en la cual encontraremos el archivo `blockMeshDict` en el cual podemos colocar el mallado. Es importante que en la parte de `boundary` tengamos claros los nombres ya que los seguiremos usando en el solver.

Vamos a la parte del solver dentro de la carpeta 0 tenemos dos archivos uno `U` y `p`, como podemos ver dentro del archivo podemos ver lo siguiente

```
boundaryField
{
    movingWall
    {
```

```

    type    zeroGradient;
}

frontAndBack
{
    type    empty;
}
}

```

Aqui es donde pondremos los nombres que escogimos en **boundary** y escogeremos alguno de los siguientes tipos para las agrupaciones de las caras.

- zeroGradient:
- symmetryPlane:
- empty:
- fixedValue:

Ahora notemos que en la parte de hasta arriba podemos encontrar lo siguiente

```

dimensions [0 2 -2 0 0 0 0];

```

aqui definimos las dimensiones del campo en este caso la presion kinematica m^2s^{-2} . Debajo de este podemos ver

```

internalField uniform 0;

```

Este quiere decir que el los datos del campo interno puede ser uniforme y descritos por un solo valor o podemos usar **nonuniform** donde todos los valores del campo deben de ser especificados

3 IncompressibleVoF

Para facilitarnos el uso de **incompressibleVoF**, importaremos el ejemplo de “Breaking of a Dam”. Al abrir la terminal, cambiamos al siguiente directorio:

```
cd run/user/OpenFOAM
```

Luego escribimos:

```
cp -r $FOAM_TUTORIALS/incompressibleVoF/damBreakLaminar .
```

3.1 Por definir

Primero, revisemos los archivos que obtenemos al copiar la carpeta.

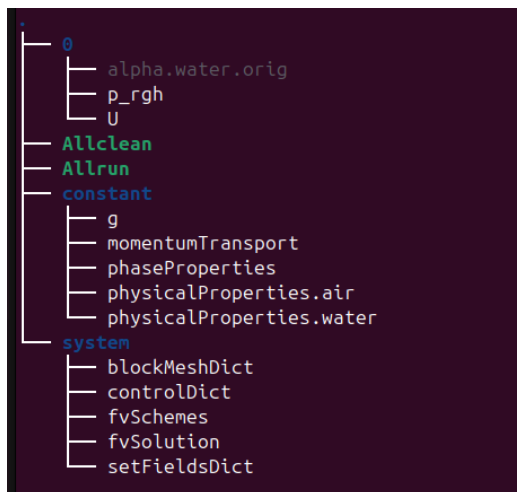


Figure 2: Archivos de damBreakLaminar

El primer archivo que vamos a editar es `system/blockMeshDict`. En este archivo se definen tres partes: `vertices`, `blocks` y `boundary`.

- En `vertices` colocamos todos los vértices de las caras de nuestro *mesh* con el formato $(x\ y\ z)$. Al terminar de colocar los puntos, estos quedarán enlistados comenzando por el 0 según su aparición en el código.
- En `blocks` definimos los bloques mediante los puntos. La sintaxis es:

`hex (p1 p2 p3 ... p8) (a b c)`

Los 8 puntos deben colocarse de manera que si existe una arista entre dos puntos, estos aparezcan consecutivos en la lista. Los valores a , b y c indican la división del bloque en cada dirección.

- En `boundary` definimos las paredes de los bloques. Los tipos más comunes son:

- **type wall**: pared sólida dentro del dominio.
- **type patch**: permite entrada y salida de fluido; no aporta información sobre la pared.
- **type symmetryPlane**: replica lo que ocurre en el plano de simetría.
- **type empty**: útil para modelar en 2D o 1D, ya que el sistema no resuelve ecuaciones en esas direcciones.

Al determinar las paredes, podemos agrupar distintas caras si tienen el mismo tipo o alguna propiedad particular que será importante más adelante. A esto se le llama **boundaryField**.

Para verificar cómo quedó el *mesh*, usamos los comandos:

```
blockMesh
paraFoam
```

blockMesh compila el código y **paraFoam** abre un visualizador para ver el *mesh*.

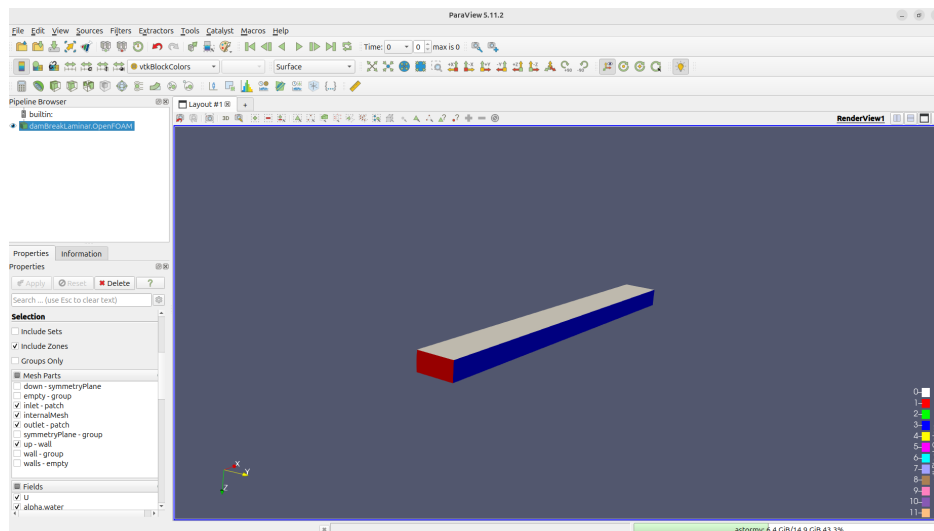


Figure 3: Modelo 3D del dominio

Ahora editaremos las condiciones iniciales del sistema, que se encuentran en la carpeta 0. Empecemos con el archivo U, que contiene las velocidades del agua en las caras del sistema (llamado **boundaryMesh**). Los tipos que se pueden asignar son:

- **fixedValue**: se asigna un valor fijo de velocidad mediante la línea `value uniform (vx vy vz)`.
- **zeroGradient**: la velocidad se obtiene de la celda vecina; se usa usualmente en las caras de salida.
- **symmetryPlane**: replica lo que ocurre en el plano de simetría.
- **noSlip**: produce paredes sin deslizamiento.
- **slip**: permite que el fluido resbale sobre la cara seleccionada.
- **empty**: no agrega ninguna condición.

También en la carpeta 0 tenemos el archivo `p_rgh`, que representa la presión hidrostática. Aquí podemos modificar los tipos de `boundaryField`. Los tipos son los mismos que en U, excepto `slip` y `noSlip`. Se añade además `fixedFluxPressure`, que es una presión fija determinada por el sistema.

Con esto solo quedaría revisar las propiedades físicas del aire y del agua. Estos archivos están en la carpeta `constant`. Podemos modificar la viscosidad (`nu`) y densidad (`rho`) del líquido y del aire en los archivos `physicalProperties.water` y `physicalProperties.air`, respectivamente. También debemos modificar `phaseProperties`, donde definimos las fases del líquido con `phases (water air)` y el coeficiente de tensión superficial (`sigma`).

Los siguientes archivos que agreguemos o modifiquemos estarán relacionados con si el flujo es laminar o turbulento.

3.2 Flujo laminar

Para flujo laminar, como el ejemplo que estamos usando ya emplea este modelo, solo mostraremos dónde se define. Abrimos la carpeta `constant` y el archivo `momentumTransport`. Allí podemos ver la variable `simulationType` con el valor `laminar`.

3.3 Flujo turbulento

Para flujo turbulento, debemos modificar el archivo `constant/momentumTransport` y establecer `simulationType RAS` (Reynolds-Averaged Simulation). Debajo de esto indicamos que usaremos el modelo `kEpsilon` y que hay turbulencia.

```
simulationType RAS;

RAS
{
    RASModel kEpsilon;
    turbulence on;
    printCoeffs on;
}
```

Figure 4: Formato del archivo RAS

Además, debemos agregar los siguientes archivos en la carpeta 0: **k**, **nut** y **epsilon**. Empecemos con **k**, donde se define la energía cinética turbulenta. En este archivo debemos especificar los **boundaryField**. Las opciones principales son:

- **fixedValue**: se asigna un valor fijo.
- **inletOutlet**: se usa cuando la dirección del flujo es desconocida.
- **symmetryPlane**
- **zeroGradient**
- **empty**

Luego revisamos **epsilon**, que representa la tasa de disipación de la energía cinética turbulenta, y tiene los mismos tipos que **k**. Finalmente, **nut** es la viscosidad turbulenta.

3.4 Agregar funciones de medición

En el archivo **system/controlDict**, al final podemos agregar funciones de medición mediante la definición de **functions** y **probes** de la siguiente manera:

Dentro de **probes**, si queremos medir la presión, hacemos lo siguiente: primero definimos el campo que nos interesa (en este caso la presión, que está en el archivo **p_rgh**) y luego indicamos los puntos donde queremos medir mediante **probeLocations**.

```

functions
{
    probes
    { ...
    }
}

DebugSwitches
{
    MULES          1;
}

```

Figure 5: Ejemplo de funciones de medición en `controlDict`

3.5 Ejecución del programa

Para ejecutar el programa usamos:

```
foamRun
```

Para visualizar los resultados:

```
paraFoam
```

Una vez dentro de ParaView, cambiamos `Solid Color` por `alpha.water` y obtenemos una visualización similar a la siguiente:

4 IncompressibleFluid

Usaremos la simulación de Backward-facing step. Para esto en la carpeta correremos lo siguiente

```
cp -r $FOAM_TUTORIALS/incompressibleFluid/pitzDailySteady .
cd pitzDailySteady
```

Ahora comparado con la simulación anterior obtenemos los siguientes archivos (colocar imagen comparando cuáles archivos faltan y cuáles agregamos). Podemos notar que en la carpeta 0 tenemos una mayor cantidad de archivos, esto pasa ya que cada uno de ellos nos permite

```

probes
{
  type          probes;
  libs          ("libsampling.so");
  writeControl   adjustableRunTime;
  writeInterval  1;

  fields
  (
    p_rgh
  );

  probeLocations
  (
    (0.5 0.25 0)
    (0.5 0.25 10)
  );
}

```

Figure 6: Función de medición de presión con `probes`



Figure 7: Movimiento del fluido (campo `alpha.water`)